

VERIGATE -- IDENTITY VERIFICATION INTELLIGENCE

Developer-Oriented Identity Verification Prototype Technical Documentation

Evidence-First Screening Pipeline | Deterministic Validation | Image Forensics | Multimodal AI Arbitration

VeriGate is a developer-oriented identity verification prototype that combines document processing, deterministic validation, image forensics, biometric face comparison, and multimodal AI reasoning into a single screening workflow.

PROTOTYPE STATUS & BOUNDARIES

VeriGate is a hackathon/MVP system validated primarily with synthetic test fixtures. It is not a production border-security deployment and does not have access to real government watchlists, passport PKI/NFC chips, or physical document security inspection hardware.

1. PROBLEM

Identity screening can involve passports, national IDs, permits, visas, and other credentials. Manual inspection can be slow and inconsistent, while isolated OCR or face checks may fail to expose contradictions across different evidence sources.

VeriGate addresses this with an evidence-first pipeline:



2. CORE ARCHITECTURE

VeriGate uses a 7-stage sequential evidence pipeline.

Stage 1 -- Input Qualification

A multimodal vision model inspects both submitted images before expensive processing begins. It checks whether the document and presented-person image are suitable for the screening workflow and can reject irrelevant or unusable inputs early.

Stage 2 -- OCR and Field Extraction

The document is processed to extract structured identity information such as name, document number, nationality, issuing country/authority, date of birth, sex/gender, date of issue (when present), and date of expiry (when present). The active project uses PaddleOCR/PP-OCR together with OpenCV, Pillow, and NumPy for image processing.

Stage 3 -- MRZ Validation

For documents containing a Machine Readable Zone, VeriGate parses the MRZ and recalculates its check digits. The stage is conditional: documents without an MRZ skip MRZ-specific checks rather than automatically failing.

Stage 4 -- Deterministic Document and Temporal Validation

This stage is rule-based. Python calculates objective evidence including: current date, date-of-birth validity, calculated age, future DOB detection, issue date, expiry date, expired/not-expired status, days until/since expiry, issue/expiry relationship, and issue date in the future.

CENTRAL ARCHITECTURAL RULE

Python calculates deterministic date facts; Gemma interprets them but must not override them.

Missing issue or expiry dates are represented as unavailable/null and are not automatically treated as invalid unless the document type requires the field.

Stage 5 -- Tampering Forensics

VeriGate performs image-based forensic analysis using Error Level Analysis (ELA). The document is recompressed, the original and recompressed images are compared, pixel residuals and variance are calculated, and a heatmap evidence artifact is produced. ELA is treated as supporting forensic evidence, not as a standalone authenticity proof.

Stage 6 -- Biometric Face Verification

The document portrait is compared with the presented person's image using DeepFace / Facenet512 facial embeddings and cosine-distance comparison. The result is returned as match, mismatch, or inconclusive evidence. If biometric evidence is unavailable or the biometric stage fails, the reasoning layer is instructed not to invent a match.

Stage 7 -- Gemma AI Arbitration

Gemma receives the available evidence from the preceding stages, including images, OCR/MRZ results, deterministic date/age evidence, tampering evidence, and biometric evidence. It produces a structured assessment containing validity, consistency, biometric status, tampering concern, risk score, risk level, decision, explanation, and risk factors.

Normal decisions are:
APPROVE
REVIEW
REJECT

3. EVIDENCE PHILOSOPHY

THE ARCHITECTURAL PRINCIPLE

Specialized systems produce evidence. AI interprets the combined evidence.

For example:

```
OCR                -> DOB = 1985-03-15
Python validation  -> Age = 41
MRZ                -> checksum = valid
Face verification  -> distance = 0.217
ELA                -> no significant anomaly
Gemma              -> cross-evidence interpretation
                  -> decision + explanation
```

This reduces the amount of deterministic work delegated to the language model and makes the system easier to debug and explain.

4. AI PROVIDER ARCHITECTURE

The AI layer is provider-configurable. The normal preference is:

```

Google AI Studio -- Gemma 4 31B
      |
      v
Ollama Vision fallback
      |
      v
OpenRouter fallback

```

The provider is selected through configuration rather than hardcoded into the rest of the pipeline. This also makes a future migration to a strong local/on-prem multimodal model possible without rebuilding the upstream verification stages.

5. DATABASE AND EVIDENCE MODEL

A screening run is represented by a central screening session with linked evidence records:

```

screening_session
|
+-- document
+-- OCR result
+-- validation result
|   +-- validation checks
+-- tampering analysis
|   +-- tampering signals
+-- face verification
+-- risk assessment
    +-- risk factors

```

The database/reference layer can support known-good reference records and future watchlist integrations.

Reference / Watchlist Scope

Real blacklist/watchlist detection requires authoritative external datasets and appropriate production query/access controls. The hackathon prototype uses synthetic/reference records and does not claim access to real border-security watchlists.

Likewise, detecting repeated identities across a population requires historical biometric/identity data and cross-session matching infrastructure; that is a future extension rather than a demonstrated production capability.

6. API FLOW

The current frontend screening flow uses the following backend endpoints:

```

POST /api/ocr/extract
|
v
Input qualification + OCR/session creation
|
v
POST /api/face/verify
|
v
Face verification
|
v
POST /api/risk/assess/{session_id}
|
v
Gemma evidence arbitration
|
v
GET /api/screening/{session_id}
|
v
Final screening report

```

ELA evidence artifacts are served from the backend evidence path.

7. FRONTEND ARCHITECTURE

The frontend is a single-page application using HTML5, CSS3, modern JavaScript, GSAP, ScrollTrigger, and the browser MediaDevices API.

```
frontend/  
+-- index.html  
+-- assets/  
|   +-- samples/  
|   +-- videos/  
+-- css/  
|   +-- variables.css  
|   +-- layout.css  
|   +-- landing.css  
|   +-- screening.css  
|   +-- styles.css  
+-- js/  
    +-- dev-guard.js  
    +-- config.js  
    +-- api.js  
    +-- animations.js  
    +-- screening.js  
    +-- app.js
```

The landing page, screening workstation, processing state, and results experience are presented as one continuous product experience.

8. SCREENING UI

The frontend supports:

- document upload
- presented-person image upload
- drag-and-drop input
- optional camera capture
- synthetic/demo sample cases
- processing visualization
- final risk report
- ELA/original comparison
- JSON audit export

The frontend is responsible for presentation and interaction; screening decisions continue to originate from the backend evidence pipeline.

9. RESULTS

A completed screening can expose:

- final decision
- risk score
- risk level
- identity fields
- calculated age
- document validation state
- MRZ state when applicable
- face match state and distance
- tampering/ELA evidence

- temporal validation
- risk factors
- AI explanation
- session/audit metadata

The aim is to expose the evidence behind the decision rather than only a binary pass/fail result.

10. TECHNOLOGY STACK

Layer	Components & Libraries
Backend	Python 3.11+, FastAPI, Uvicorn, Pydantic v2, HTTPX
Vision & Forensics	OpenCV, Pillow, NumPy, Error Level Analysis (ELA)
OCR Engine	PaddleOCR / PP-OCR
Biometrics	DeepFace, Facenet512, Cosine distance comparison
Database	Supabase / PostgreSQL (where configured)
AI Layer	Google AI Studio -- Gemma 4 31B (gemma-4-31b-it)
AI Fallbacks	Ollama Vision (qwen2.5-vl), OpenRouter fallback
Frontend	HTML5, CSS3, JavaScript, GSAP, ScrollTrigger, MediaDevices API

11. INSTALLATION

Prerequisites

Install Python 3.11+, Git, the project dependencies, and at least one configured AI provider.

Clone the repository:

```
git clone https://github.com/mainSiddharthhoon/Veri-Gate.git
cd Veri-Gate
```

Create and activate a virtual environment:

Windows PowerShell:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Linux / macOS:

```
python -m venv venv
source venv/bin/activate
```

Install Python dependencies:

```
pip install -r backend/requirements.txt
```

If the active OCR installation requires an external OCR runtime, install the required runtime for the chosen environment.

12. ENVIRONMENT CONFIGURATION

Create .env in the project root and provide the values appropriate for the selected deployment.

```
APP_NAME=VeriGate
APP_VERSION=0.1.0
DEBUG=false
HOST=127.0.0.1
PORT=8000

SUPABASE_URL=your-supabase-url
SUPABASE_ANON_KEY=your-supabase-anon-key
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key

GEMINI_API_KEY=your-google-ai-studio-key

VISION_BASE_URL=http://127.0.0.1:11434
VISION_MODEL=qwen2.5-vl-7b-local
VISION_API_KEY=

OPENROUTER_API_KEY=your-openrouter-key
OPENROUTER_MODEL=openrouter/free
```

SECURITY RULE

Never commit real API keys or other secrets.

13. RUNNING VERIGATE

From the project root:

```
python -m uvicorn app.main:app --app-dir backend --host 127.0.0.1 --port 8000
```

Open the web application at:

```
http://127.0.0.1:8000/frontend/
```

Useful endpoints:

```
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/api/health
```

14. TESTING

Run the backend test suite with:

```
pytest backend/tests -v
```

Important test areas include:

```
pytest backend/tests/test_mrz.py -v
pytest backend/tests/test_validation.py -v
pytest backend/tests/test_field_extraction.py -v
pytest backend/tests/test_tampering.py -v
pytest backend/tests/test_face.py -v
pytest backend/tests/test_risk.py -v
```

The project was also validated with controlled synthetic end-to-end cases covering valid identities, biometric mismatches, invalid inputs, temporal inconsistencies, future DOBs, and multiple-error documents.

15. SYNTHETIC TESTING DATA

The repository uses synthetic fixtures for development and demonstration. Examples include:

```
valid/  
  matching identity cases  
  
invalid/  
  invalid input  
  expired documents  
  future DOB  
  temporal contradictions  
  biometric mismatches  
  multiple-error documents
```

Documents branded VERIGATE / SYNTHETIC IDENTITY LAB are authorized synthetic fixtures for this application's testing workflow.

PRIVACY & ETHICAL COMPLIANCE

No real government identity documents or real personal identity datasets are intended to be distributed with the prototype.

16. SECURITY AND PRIVACY CONSIDERATIONS

The prototype is designed so the AI reasoning layer can eventually be moved entirely to private infrastructure.

A production deployment should additionally implement:

- authenticated operators
- role-based access control (RBAC)
- encrypted transport (TLS 1.3)
- encrypted storage at rest
- secrets management
- retention and deletion policies
- controlled access to biometric evidence
- dataset governance
- production monitoring and audit controls

17. KNOWN LIMITATIONS

Physical security features

A normal image pipeline cannot reliably inspect tactile engraving, UV-only features, IR-only features, physical holograms, optically variable inks, or other features requiring specialized inspection hardware.

NFC / ePassport cryptography

The prototype does not read or cryptographically verify passport NFC chips, ePassport PKI signatures, or chip-side biometric data.

Lighting and occlusion

Glare, shadows, blur, poor face crops, masks, sunglasses, and other image-quality problems can make biometric or forensic stages inconclusive.

Document coverage

MRZ validation is conditional on an MRZ being present and parsable. Documents without an MRZ use OCR and deterministic validation instead.

Dataset-dependent intelligence

Real blacklist/watchlist detection and cross-session multiple-identity detection require authoritative datasets and additional matching infrastructure.

AI latency

Cloud AI latency depends on network/provider conditions. Local multimodal model latency depends heavily on available CPU, GPU, RAM, and model configuration.

Prototype evaluation

Current demonstrations use controlled synthetic fixtures. These tests should not be presented as production accuracy benchmarks.

18. FUTURE EXTENSIONS

Possible next steps include:

- authoritative watchlist/blacklist integration
- cross-session identity correlation
- broader document-type support
- specialized visa/stamp validation
- stronger document-security inspection
- NFC/ePassport verification
- stronger local/on-prem multimodal reasoning
- production authentication and RBAC
- labeled-dataset benchmarking
- large-scale monitoring and analytics

19. DESIGN RATIONALE

The system deliberately avoids asking the LLM to perform every verification task.

Deterministic code	-> mathematical/date facts
OCR / MRZ engines	-> document evidence
Specialized vision/forensics	-> biometric and image evidence
Gemma	-> cross-evidence reasoning
	-> risk + decision + explanation

This separation makes the pipeline easier to reason about, test, debug, and upgrade. A future local/on-prem multimodal model can replace the current AI provider while preserving the upstream verification architecture.

20. CURRENT PROTOTYPE SCOPE

Implemented

- multimodal input qualification
- OCR and structured field extraction
- conditional MRZ validation
- deterministic document validation
- deterministic date/age validation
- ELA-based tampering evidence
- biometric face verification
- multimodal AI evidence arbitration
- risk scoring
- approve/review/reject decisions
- explainable screening report
- camera capture
- ELA/original comparison
- JSON audit export
- modular single-page frontend
- synthetic end-to-end testing fixtures

Not claimed by this prototype

- real government watchlist access

- production border deployment
- NFC/ePassport chip verification
- physical security-feature inspection
- universal document recognition
- guaranteed production accuracy
- guaranteed fixed latency
- regulatory certification

21. SUMMARY

VeriGate is an evidence-first identity screening system:

```
Input
-> Qualification
-> OCR
-> MRZ (when applicable)
-> Deterministic Validation
-> ELA Forensics
-> Face Verification
-> Gemma Reasoning
-> Risk + Decision + Explanation
```

The architecture is modular so individual verification components and the AI reasoning provider can be improved or replaced without rebuilding the entire application.

22. LICENSE

See the repository LICENSE file for the applicable license terms.

END OF DEVELOPER DOCUMENTATION -- VERIGATE PROTOTYPE